CS 0007: Introduction to Java Lecture 12

Nathan Ong
University of Pittsburgh
October 20, 2016

Annoucements

- Project 0 submission link is now available via CourseWeb
- When submitting, please organize your source code and your essay into one zip file. The zip file name should be LastnameFirstname_Project0.zip
- Please DO NOT include any *.class files

```
Type[] name = new Type[size];
```

```
int[] empty = new int[10];
```

```
int[] list = {1,2,3,4};
```

while(){…}

- While a condition is true, run the block
- While not at end of the list, print out the next element

```
while(not at end of list)
{
        System.out.println(next element);
}//end loop while(not at end of list)
```

while(){…}

```
int currElement = 0;
while(currElement < list.length)
{

  System.out.println(list[currElement]);
      currElement++;
}//end loop
  while(currElement<list.length)
```

So Many Things We Have to Remember...WHYYYYYYYYYY

```
int counter = start;
while(condition)
{
        //Do stuff
        counter increment
}//end loop while(condition)
```

- There must be a better way
- WILD KEYWORD APPEARS

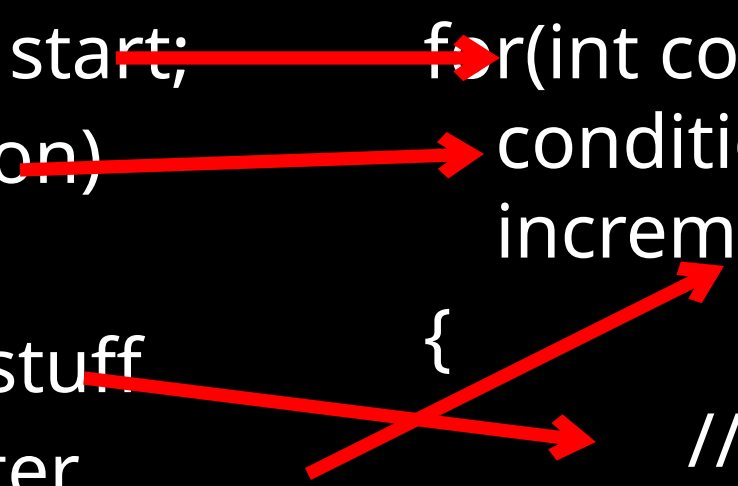
`for(;;){…}`

- The for loop allows us to put everything together for us!
- for(counter = start; condition; counter increment)
- Now we don't have to worry about the placement of the counter and incrementing it!

EQUIVALENCE

```
int counter = start;
while(condition)
{
        //Do stuff
        counter
   increment
}//end loop
  //while(condition)
```

```
for(int counter = start;
   condition; counter
   increment)
{
        //Do stuff
}//end loop
   for(condition)
```



for(;;){…}

```
for(int currElement = 0; currElement <
  list.length; currElement++)
{

  System.out.println(list[currElement]);
}//end loop for(currElement <
  list.length)
```

A Note

- Programmers are lazy
- Therefore, **`currElement`** is too long for a name
- Conventional counter names are: **`i`**, **`j`**, **`k`**, **`l`**, **`m`**, **`n`**, **`c`**, **`a`**, **`b`**, **`d`**, **`num`**, **`count`**, **`counter`**

for(;;){…}

```
for(int i = 0; i < list.length; i++)
{
      System.out.println(
          list[currElement]);
}//end loop for(i < list.length)
```

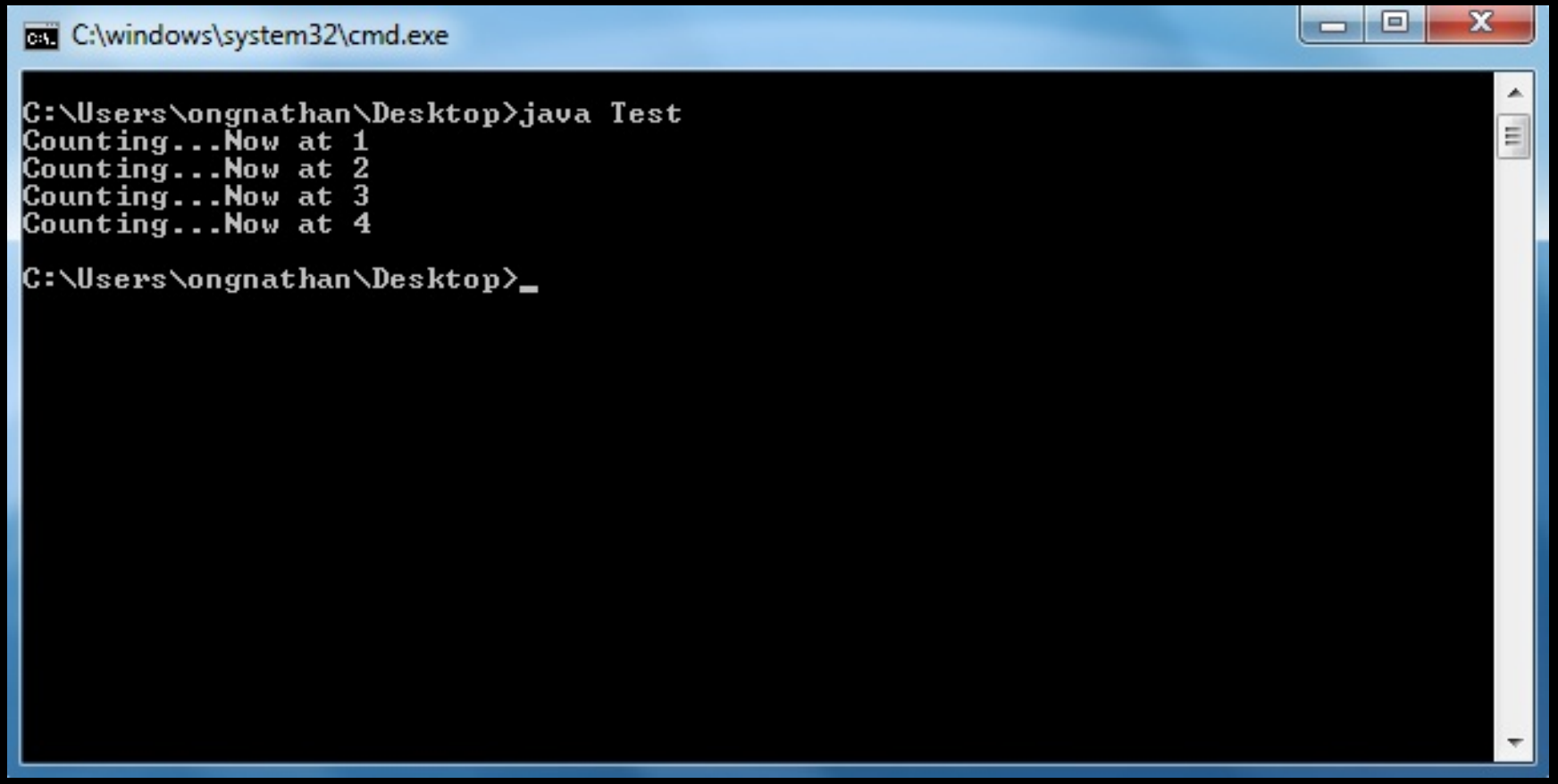
Let's Test It on List

```
int[] list = {1,2,3,4};
```

Let's Test It on List

```
public class Test
{
     public static void main(String[] args)
     {
         int[] list = {1,2,3,4};
         for(int i = 0; i < list.length; i++)
         {
         System.out.println("Counting...    Now at " +
   list[i]);
         }//end loop for(i < list.length)
     }//end method main
}//End class Test
```

What is the output?



It works, YAAAAAYYYYYY!!!

Usage

- When do I use For Loops?
  - Iteration, or going through all of the elements of an array or other type of list.
  - When the number of times you need to loop is known, either explicitly with a value, or through a variable.
- While loops are used otherwise

Multi-dimension Arrays

- Arrays don't just have to be lists
- They can also be grids, cubes, hyper-cubes, ...

```
Type[]...[] name = new Type[size1]...[sizeN];
```

```
Type[]...[] name = values...s;
```

```
int[][] listList = {{1,2,3},
    {4,5,6},{7,8,9}};
```

Things to Remember

- In 2-D Arrays, referencing an element requires TWO PAIRS of square brackets
- To get the #1 from listList
- **`listList[0][0]`**
- To get the #4 from listList
- **`listList[1][0]`**
- Remember: element listList[row][col]

How Do We Go Through a 2-D Array?

- If it took one for loop to go though a 1-D array, then...
- It will take two for loops to go through a 2-D array
- It will take N for loops to go through an N-D array
- If you don't have to go through everything, don't. It just wastes time

We need to modify this

```
public class Test
{
     public static void main(String[] args)
     {
     int[][] listList =
     {{1,2,3},{4,5,6},{7,8,9}};
     for(int i = 0; i < listList.length; i++)
     {
     System.out.println("Counting...   Now at " + listList[i]);
     }//end loop for(i < list.length)
     }//end method main
}//end class Test
```

There is only one loop. How do we write the next one?

Inner For Loop

```
for(int i = 0; i < listList.length;
  i++)
{
  //inner for loop
  //…
  //end inner for loop
}//end outer-loop for(i <
  //listList.length)
```

What does the outer for loop go through?

What should the inner for loop go through?

listList[row][col]

Inner For Loop

```
for(int i = 0; i < listList.length;
  i++)
{
  //inner for loop
  //…
  //end inner for loop
}//end outer-loop for(i <
  //listList.length)
```

The outer loop goes through the row

The inner loop goes through the columns

Inner For Loop

```
//inner for loop
for(int i = 0; i < listList.length;
  i++)
{
  System.out.println("Counting…
      Now at " + listList[i][i]);
}//end inner-loop for(i <
  //listList.length)
```

Is this allowed?

No because i is already used

Inner For Loop

```
//inner for loop
for(int j = 0; j < listList.length; j++)
{
      System.out.println("Counting…     Now at "
  + listList[i][j]);
}//end inner-loop for(j <
  //listList.length)
```

Is this allowed?

No because **`listList.length`** only gets the number of rows!

What can we do?

What if we use each individual row's length?

Inner For Loop

```
//inner for loop
for(int j = 0; j <
  listList[i].length; j++)
{
  System.out.println("Counting…
      Now at " + listList[i][j]);
}//end inner-loop for(j <
  //listList[i].length)
```

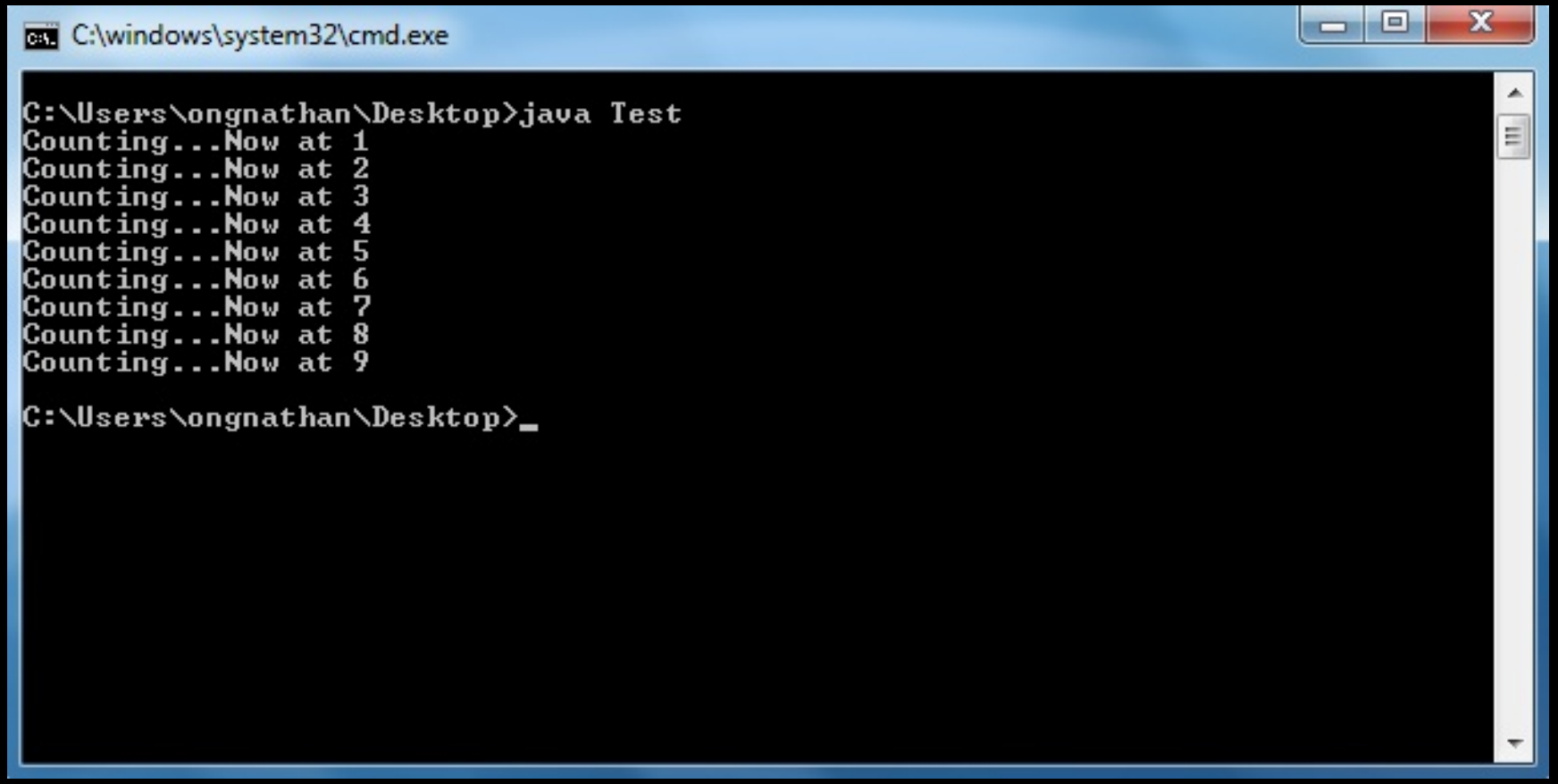
Does this work?

Let's Test It on listList

```
public class Test
{
     public static void main(String[] args)
     {
         int[][] list = {{1,2,3},{4,5,6},{7,8,9}};
         for(int i = 0; i < list.length; i++)
         {
             for(int j = 0; j < list[i].length; j++)
             {
                 System.out.println("Counting...
                     Now at " + list[i][j]);
             }//end inner-loop for(j < list[i].length)
         }//end outer-loop for(i < list.length)
     }//end method main
}//End class Test
```

For convenience sake, listList list

What is the output?



It works, YAAAAAYYYYYY!!!